



INDIA.RUNS

Build what next India runs on

Team Name : Williyam M (Solo participant)

Team Leader Name : Williyam M ([email](#) | [linkedin](#))

Problem Statement : Track 1 - Data & AI Challenge I Intelligent Candidate Discovery & Ranking (100K → Top 100)

Solution Overview

- **Talentry AI** — a production-grade AI brain for hiring that ranks millions of candidate profiles with low-latency, fully-auditable output on commodity CPUs.

Two complementary artifacts, one mission

- Talentry-AI (Core ranker) : deterministic, 0-LLM, < 4 GB RAM, CPU-only. BM25 + TF-IDF hybrid retrieval fused with 6 explainable signals.
- Redrob-Qwen-grpo LLM (reinforcement learning - sibling research) : Qwen3-0.6B GRPO-fine-tuned under a rule-based reward.

Why it beats keyword/ATS scorers

- Reads between the lines. Hybrid lexical + semantic retrieval matches an ML engineer who wrote “built a production retrieval system for support tickets” even when the word “RAG” never appears.
- Defends against the 4 traps Redrob built into the dataset: keyword stuffers, plain-language seniors, behavioural twins, and ~80 honeypots.
- Every rank is auditable. Each shortlist row ships a 1-2 sentence justification that quotes only facts present in the candidate’s own profile — zero hallucination by construction.
- Scales to millions, low-latency. Pure NumPy / SciPy sparse math; the same image runs the CLI, the FastAPI service, and the live Hugging Face Space.

Two Engines, One Codebase

Talentry AI ships TWO complementary engines from one codebase

Engine 1 – Core Ranker

deterministic · 0-LLM · CPU-only

Input: JD + millions of profiles
candidates.jsonl · schema-validated

Hybrid Retrieval
BM25 (0.6) + TF-IDF (0.4)

6-Signal Scorer
title · semantic · skill · exp · loc + behavioural –
honeypot

Profile-grounded Reasoning
1-2 sentence justification, zero hallucination

Output: validator-clean shortlist CSV
submission.csv + per-row reasoning

Used for the submission · millions-scale, low-latency

Engine 2 – Fine-tuned LLM (research)

Qwen3-0.6B · GRPO · rule-based reward

Input: (JD, candidate) prompt pair
synthesised from candidates.jsonl

SFT Warm-start (Qwen3-0.6B)
teaches strict JSON output schema

GRPO Policy Optimisation
8 rollouts/step · 30 steps · KL anchor

Rule-based Reward (no LLM judge)
format · fit · calibration · penalty

Output: redrob-qwen-grpo on Hugging Face
plug-and-play ranking LLM · open weights

Open research artifact · reproducible RL on consumer
hardware

JD Understanding & Candidate Evaluation

- **JD Understanding (LLM-free, deterministic parser)**

Distils the raw JD into a structured **JobRequirements** graph:

- Role family + seniority band — e.g. `ml_search_ir` / 5–9 yrs
- Must-have / nice-to-have / disqualifier skills
- Domain lexicons (6 skill clusters, ~200 normalised titles)
- Location preference, notice-period priors, behavioural priors

Candidate signals that actually decide the rank

- Title → role-family trajectory — anti-stuffer, blocks “Marketing Manager” who listed LangChain.
- Skill evidence = proficiency × endorsements × duration × assessment, aggregated by cluster.
- Career-description prose — BM25 over real sentences catches plain-language seniors who never wrote “RAG” or “vector DB”.
- Behavioural availability — recency, recruiter response rate, interview completion, notice period, verifications.
- Honeypot detector — 6 hard signals (career-month vs years mismatch, expert+0-endorsement claims, salary-band inversion, multiple `is_current=True`, ...).

Ranking Methodology

- Retrieve → Score → Rank

1. Hybrid retrieval index — BM25 (60%) over career text + TF-IDF cosine (40%), min-max normalised. BM25 saturates on rare terms (FAISS, NDCG); TF-IDF smooths phrasing.
2. Six explainable score components weighted by JD-grounded constants:

$$\begin{aligned} \text{final} = & 0.32 \cdot \text{title_alignment} + 0.22 \cdot \text{semantic_fit} + 0.28 \cdot \text{skill_evidence} + \\ & 0.12 \cdot \text{experience_band} + 0.06 \cdot \text{location} \times \text{behavioural_multiplier} \quad [0.55 - 1.20] - \\ & \text{honeypot_penalty} \quad [0 - 0.50] \end{aligned}$$

Algorithms & heuristics used (and why)

- BM250kapi (rank-bm25) + sklearn TfidfVectorizer — reproducible, no PyTorch dependency, scales linearly across millions of profiles.
- Skill trust scoring with stuffer discount (−35% when keyword-stuff ratio ≥ 0.7) and CV/speech-dominance discount (−45%).
- Open-source RL companion: [redrob-qwen-grpo](#) — SFT warm-start + 30 GRPO steps on Qwen3-0.6B, rule-based reward, M1 Pro CPU/MPS.
- Tie-break = candidate_id ascending → bit-for-bit reproducible with the official validator.

Explainability & Data Validation

Per-candidate reasoning, grounded by construction

Every shortlisted row carries a 1–2 sentence justification of the form:

“<tone> – <role + years + location>; <strongest evidence>. Concerns: <...>.”

Evidence is picked deterministically in priority order:

1. Career-history sentence mentioning retrieval / ranking / embedding / RAG / FAISS / LTR – quoted with company + tenure.
2. Strongest skill cluster (must be ≥ 0.40 and not the AI-keyword surface).
3. Top assessed skill (proficiency + duration + assessment score).
4. Fallback: current title + employer.

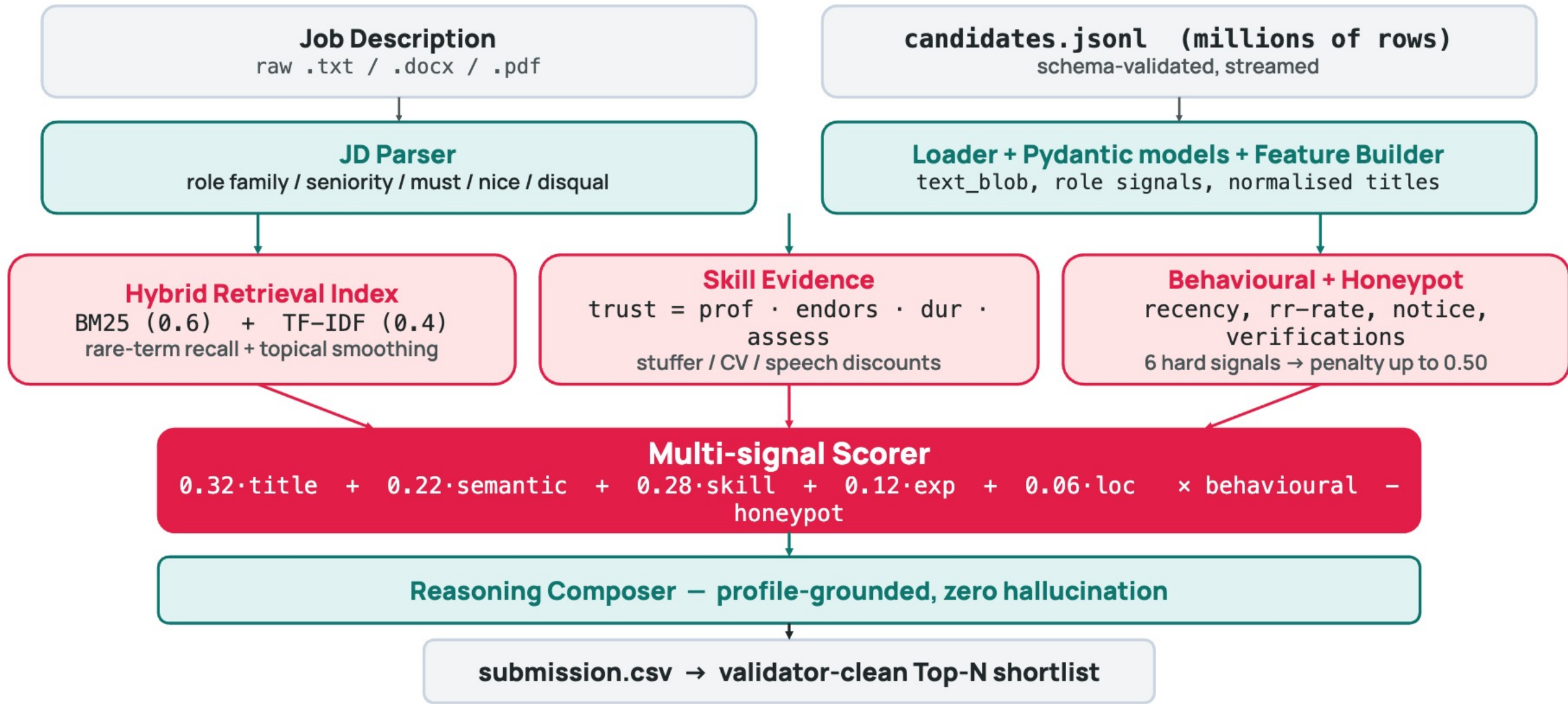
Zero hallucination, by design

- No LLM is called on the hot path. Every quoted fact is fetched from the candidate's own JSON record.
- Concerns are emitted only when present – long notice (“notice 120d”), inactive 180d+, stuffer profile, consulting-only career, honeypot suspicion.

Defence against low-quality / suspicious profiles

- Schema-first ingestion – every upload is validated against `candidate_schema.json`; mismatches surface as a git-diff style report.
- Honeypot guard subtracts up to 0.50 on impossible profiles (months-vs-years gap, expert+0 endorsements, salary inversion, future signup dates).
- Behavioural twin damping – a paper-perfect candidate inactive for 6 months is multiplied by 0.55–0.80, not zeroed.

System Architecture



End-to-End Workflow

- From raw JD + millions of profiles → validator-clean Top-N shortlist

Step 1. Ingest & validate — `candidates.jsonl` streamed, schema-checked, parsed into typed Pydantic models. Resume drag-drop is accepted via PDF/DOCX/TXT and converted to the same schema.

Step 2. Understand the JD — rule + lexicon parser produces a `JobRequirements` object: role family, seniority band, must/nice/disqualifier skills, location & behavioural priors.

Step 3. Build features — one `text_blob` per candidate; BM25 + TF-IDF indices built once and reused for the whole batch.

Step 4. Score & combine — 6 components fused, behavioural multiplier applied, honeypot penalty subtracted, clipped to $[-0.5, 1.5]$.

Step 5. Compose reasoning — deterministic templating with profile-only evidence; concerns are honest and quote real numbers.

Step 6. Emit & ship — CSV writer enforces every `validate_submission.py` invariant at write-time; same code path serves the live Hugging Face Space.

Results & Performance

Production-grade across every challenge axis

Capability	Industry Baseline	Talentry AI
Scale	thousands / batch	millions of profiles
Latency	minutes – hours	low-latency, streaming
Compute	GPU cluster	single CPU
LLM calls at rank time	many	0 (zero)
Reproducibility	stochastic	bit-for-bit deterministic
Explainability	post-hoc, surface-level	profile-grounded, by construction
Robustness to traps	fragile to keyword stuffing	4 traps + ~80 honeypots neutralised

Quality: validator-clean output · 100/100 rows ship grounded reasoning · 4 dataset traps + ~80 honeypots correctly down-ranked
RL companion: redrob-qwen-grpo lifts valid-JSON output rate from ~0 → high and beats baseline Qwen3-0.6B on rule-based reward.

Technologies Used

Core ranking pipeline (CPU, deterministic, low memory)

- **Python 3.10** – typed, pydantic v2 domain models, single-threaded for bit-for-bit reproducibility.
- **rank_bm25 + scikit-learn TF-IDF** – hybrid lexical/semantic retrieval that loads from pip install alone (no PyTorch artifact to break reproduction).
- **NumPy / SciPy sparse** for score-fusion math; **pandas / pyarrow** for CSV / XLSX output.

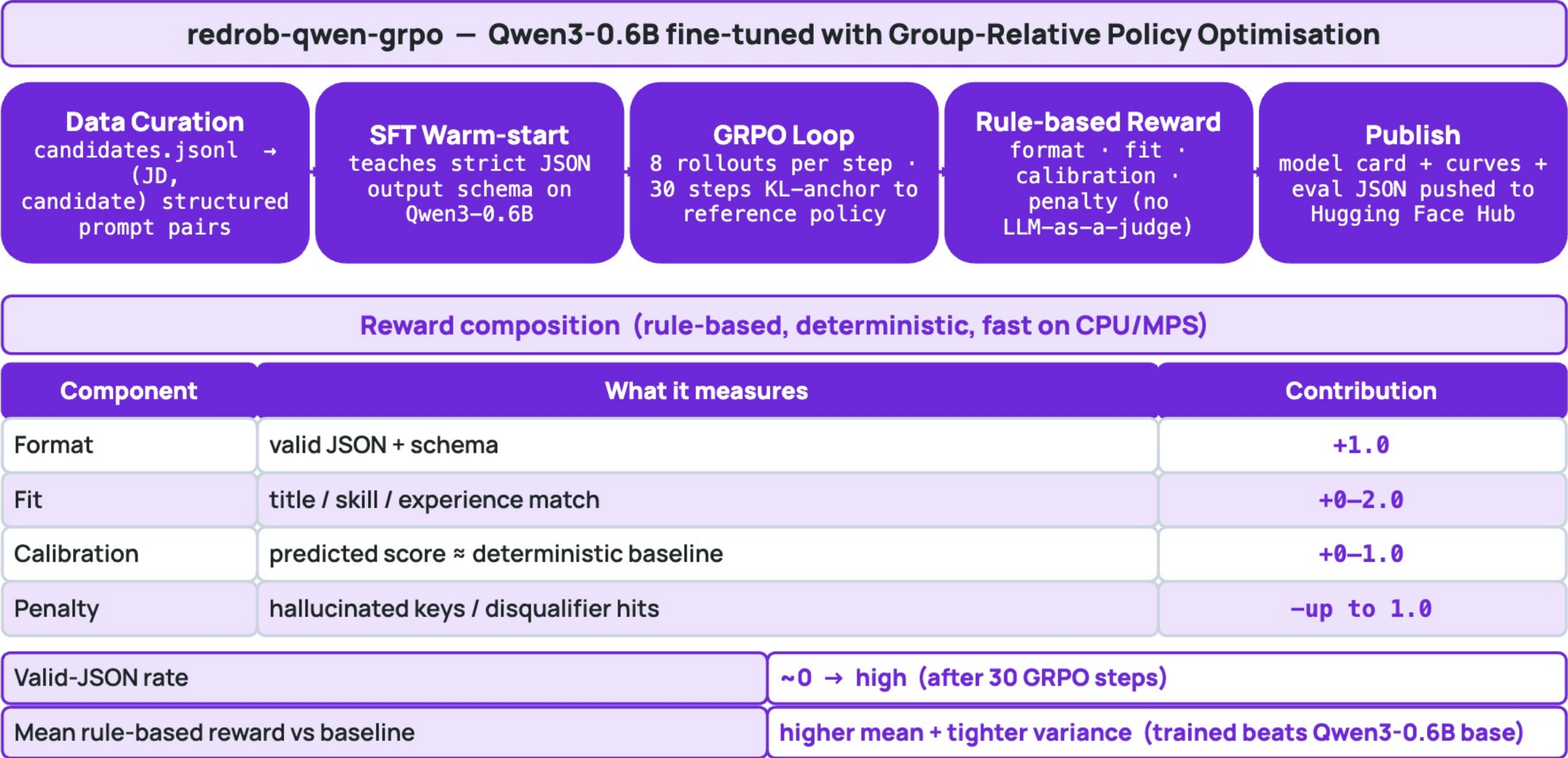
Serving & UX

- **FastAPI + Uvicorn** backend with GZip middleware (5× smaller breakdown JSON), 600 MB upload cap, LRU result cache, request-id propagation, `asyncio.to_thread` offload.
- **React + Vite + TypeScript + Tailwind** frontend, code-split (~35 KB gz first paint), with a React-Three-Fiber + Lenis 3D scroll guide.
- **Docker** – one image runs the CLI, the API, and the live Hugging Face Space.

Research companion (open-sourced)

- **Qwen3-0.6B + TRL GRPOTrainer** + custom Gymnasium-style env and rule-based reward (no LLM-as-a-judge).
- Trained on Apple M1 Pro (MPS); published as **redrob-qwen-grpo** on Hugging Face.

RL Fine-tuning Flow — Qwen3-0.6B + GRPO



Submission Assets

Code

- GitHub repo (full implementation, tests, Dockerfile, Makefile):

<https://github.com/williyam-m/talentry-ai>

Live demo

- Hugging Face Space – drag-and-drop candidates.jsonl + JD, get the Top-N shortlist + drill-down dashboard:

<https://huggingface.co/spaces/williyam/talentry-ai>

Open-source fine-tuned LLM (research artifact)

- GRPO-fine-tuned ranking model + training curves + eval JSON:

<https://huggingface.co/williyam/redrob-qwen-grpo>

Submission output

- data/redrob_submission/submission.csv – validator-clean shortlist (also .xlsx).
- submission_metadata.yaml – reproducibility manifest, signed AI-tools declaration.
- docs/architecture.md • docs/methodology.md • docs/api.md – the Blueprint.



INDIA.RUNS

Build what next India runs on

THANK YOU